

kernel-talk — Speaker Notes

Lee O'Saul: Print this. Glance. Keep talking.

SLIDE 1 · ~1 min

Title

Hook

Open with the quote on screen.

SAY:

"The Linux kernel is 40 million lines of code. When something breaks at runtime, you have grep, you have gdb, and you have a lot of suffering."

"kernel-talk is an attempt to fix that — a system that knows the source, the binary, the live symbols, and live memory, all at once."

"Let me walk you through how it works and what we found."

SLIDE 2 · ~2 min

Problem & Motivation

40M+ lines

No IDE fully understands it. ctags/cscope break on the macro soup.

Four layers

Source, binary, live symbols, live memory — each needs a different tool.

Existing tools

grep = source only. addr2line = binary only. crash/gdb = runtime only.

The real question

"Why is this struct field corrupted and which commit introduced it?" — no single tool answers this.

SAY:

"Every layer of the kernel speaks a different language. Source is C. The binary is DWARF. The running kernel speaks kallsyms addresses. And live memory you can only read with drgn."

"kernel-talk fuses all four into one graph you can query in plain English."

SLIDE 3 · ~2 min

Research Questions

RQ1

Can one graph index unify all four layers?

RQ2

Does domain fine-tuning beat a general encoder on kernel retrieval?

RQ3

Can we build a training dataset automatically from git history + tree-sitter?

RQ4

Can Graph-RAG answer runtime questions without attaching to the kernel?

SAY:

"These are the four questions this project set out to answer."

"Spoiler: yes to all four — and I will show you the numbers."

SLIDE 4 · ~2 min

Related Work

DPR / CoBERT

Dense retrieval for text — we adapt to C code.

CodeBERT / StarCoder

Pre-trained on GitHub — not on kernel semantics.

Graph-RAG (Microsoft 2024)

RAG + knowledge graph for NLP — we extend to typed AST graphs.

crash / drgn / gdb

Runtime-only — no retrieval, no synthesis.

Gap

Nothing unifies all four layers with a retrieval pipeline. This is new.

SAY:

"Prior work exists in pieces. Dense retrieval, code models, graph-augmented RAG — but nobody has put them together for the kernel across all four representation layers."

"This is a genuinely new problem and a new solution."

SLIDE 5 · ~2 min

System Architecture

Layer 1

C source → tree-sitter AST → CodeNode → ChromaDB + KernelGraph

Layer 2

ELF binary → DWARF → BinarySymbol, StructLayout, addr2line

Layer 3

/proc/kallsyms → KASLR slide → live address per function

Layer 4

/proc/kcore via drgn → live struct field values, no halt

Training

synth.py → BM25 enrichment → curriculum DataLoader → DistilBERT fine-tune

SAY:

"Think of this as a digital twin of the kernel. Each layer adds a new dimension of understanding."

"Layer 1 is always available — just needs the source. Layers 2-4 need a live Linux host, which this machine is."

SLIDE 6 · ~2 min

Dataset: Kernel Mirror Index

Source

/usr/src/linux-* on the host machine — unique per machine and kernel version.

Collection

tree-sitter parse → CodeNode extraction → embed → graph edges. Under 5 minutes.

Composition

5,755 nodes | 8,843 edges | 6,623 training triplets.

Motivation

No kernel retrieval benchmark exists. CodeSearchNet has no kernel coverage.

Recommended use

Retrieval, Graph-RAG synthesis, bi-encoder fine-tuning, cross-layer debugging.

SAY:

"This is also a dataset contribution. Every machine that runs kernel-talk generates its own dataset — machine-specific, version-specific, patch-specific."

"The tool is the dataset pipeline."

SLIDE 7 · ~1 min

Mirror Index — Live Build Statistics

Numbers

5,755 CodeNodes | 8,843 edges | 768-dim ChromaDB embeddings | O(1) edge resolution

Note

These are from THIS machine, THIS kernel — live production numbers.

SAY:

"These are not synthetic numbers from a paper. This is what the indexer produced on this laptop from the running CachyOS kernel."

SLIDE 8 · ~2 min

Training Pipeline

Stage 1a

mine.py — git history → commit messages → changed functions.

Stage 1b

synth.py — 3 strategies: symbol name, docstring, caller-callee.

Stage 2

bm25.py — hard negative enrichment with C-aware tokenizer. Median gap 6.65.

Stage 3

dataset.py — curriculum DataLoader, easy-first adaptive hard ratio.

Stage 4

train_biencoder.py — InfoNCE loss, gradient accumulation x8, 3 epochs.

SAY:

"With no labeled kernel retrieval data, we had to build the dataset ourselves — synthetically, from the code itself."

"Three strategies, 6,623 triplets, BM25 hard negatives to make the contrastive learning actually challenging."

SLIDE 9 · ~2 min

Bi-Encoder Model & Training

Backbone

DistilBERT 66M params — fast enough to train on CPU in reasonable time.

InfoNCE loss

Each query sees 255 negatives per step — in-batch negatives, free.

Curriculum

$\text{effective_ratio}(p,d) = r_hard * [p + (1-p)(1-d)]$ — hard examples ramp in gradually.

Hardware

Intel Core Ultra 7 164U, 14 cores, OMP_NUM_THREADS=14, CPU-only.

Result

Val Recall@10 = 0.764 at epoch 3. Best checkpoint saved.

SAY:

"We chose DistilBERT because it is 40% smaller than BERT with 97% of the performance, and we are running on CPU."

"The curriculum ensures the model sees easy examples first — important with only 6,623 training samples."

SLIDE 10 · ~2 min

Retrieval & Synthesis Pipeline

Query

Natural language → CodeEmbedder → ChromaDB top-k → KernelGraph BFS → HybridResult

[THEORY]/[REALITY]

Top code nodes (75% token budget) + live drgn values (25%).

LLM backends

ollama (local), openai, anthropic — swappable via provider string.

Key property

Works fully offline with a local Ollama model. No cloud dependency.

SAY:

"The [THEORY] vs [REALITY] split lets the model flag when source code and live memory disagree — exactly where bugs hide."

SLIDE 11 · ~1 min

Native Rust CLI Binary

Zero-startup

ktalk version / status / env run natively — no Python startup.

Delegation

ktalk ask / index / train / eval delegate to Python subprocess.

Build

cargo build --release or make ktalk-bin

SAY:

"Single binary, no Python environment needed for basic operations."

SLIDE 12 · ~2 min

Results & Next Steps

Val Recall@10

0.764 at epoch 3 — up from near-zero. Kernel fine-tuning works.

Training curve

Ep1: 0.447 | Ep2: 0.471 | Ep3: 0.764 — big jump when curriculum fully engages.

Infrastructure

134 tests passing | 5,755 nodes | 8,843 edges | 6,623 triplets | native Rust binary.

Baseline

General-purpose DistilBERT before fine-tuning: near-zero Recall@10 on kernel queries.

SAY:

"0.764 means: for 3 out of 4 kernel questions, the right function is in the top 10 results."

"This is on a laptop CPU with no GPU. Headroom with proper hardware is significant."

Discussion & Conclusions

Finding 1

Kernel-specific fine-tuning works — 0.764 Recall@10 on CPU in 3 epochs.

Finding 2

Synthetic data is viable — auto-generated triplets match manual quality.

Finding 3

Multi-layer unification is tractable — typed graph, $O(1)$ lookup.

Finding 4

The dataset is a contribution — every deployment generates a new machine-specific dataset.

Implication

Graph-RAG can replace heavyweight debuggers for code comprehension tasks.

SAY:

"The answers to all four research questions are yes."

"The broader implication: this approach could give kernel developers much better tools for understanding complex systems software."

Future Work

Cross-encoder reranker

Stage 5 — margin loss on top of bi-encoder for better precision.

Full eval

Re-index kernel/, mm/, fs/ — measure nDCG@10 and MRR on 142 gold queries.

Kernel code editor

IDE backend — semantic nav, cross-layer call graph, inline synthesis.

XPU acceleration

Intel Arc GPU via IPEX for faster training and embedding.

Full kernel coverage

Expand from include/ (5,755 nodes) to all 40M+ lines.

SAY:

"The long-term vision is a kernel code editor — not just Q&A;, but an IDE that understands the kernel at all four layers simultaneously."

"Thank you."